

252-0027

Einführung in die Programmierung
Übungen

Woche 4: Types, Schleifen, Methoden

Fabian Rohlik

Departement Informatik

ETH Zürich

Organisatorisches

- Mein Name: Fabian Rohlik
- Bei Fragen: frohlik@student.ethz.ch
- Notizen: cs.rohlik.net, dann auf EProg Übungsstunde klicken
- Neue Aufgaben: **Dienstag Abend** (im Normalfall)
- Abgabe der Übungen bis **Dienstag Abend (23:59)**
Folgewoche

Feedback von u03

- persönliches Feedback folgt wie letzte Woche in der FEEDBACK.md Datei
- sehr gut gelöst, viele von euch auch schon über dem aktuellen Niveau der Vorlesung
- »eine beliebige Ausgabe« heißt, *ihr* könnt entscheiden; es muss nicht actually zufällig sein

Heute

- Primitive Typen
- Schleifen
- Increment und Decrement
- Git Merge Conflicts
- Methoden
- Strings
- Nachbesprechung

Primitive Typen in Java

primitive Typen («primitive types»):

für Zahlen, Buchstaben und Wahrheitswerte. z.B.

<u>Name</u>	<u>Beschreibung</u>	<u>Beispiele</u>
int	ganze Zahlen	-2147483648, -3, 0, 42, 2147483647
long	grosse ganze Zahlen	42, -3, 0, 9223372036854775807
double	reelle Zahlen	3.1, -0.25, 9.4e3
char	(einzelne) Buchstaben	'a', 'X', '?', '\n'
boolean	Wahrheitswerte	true, false

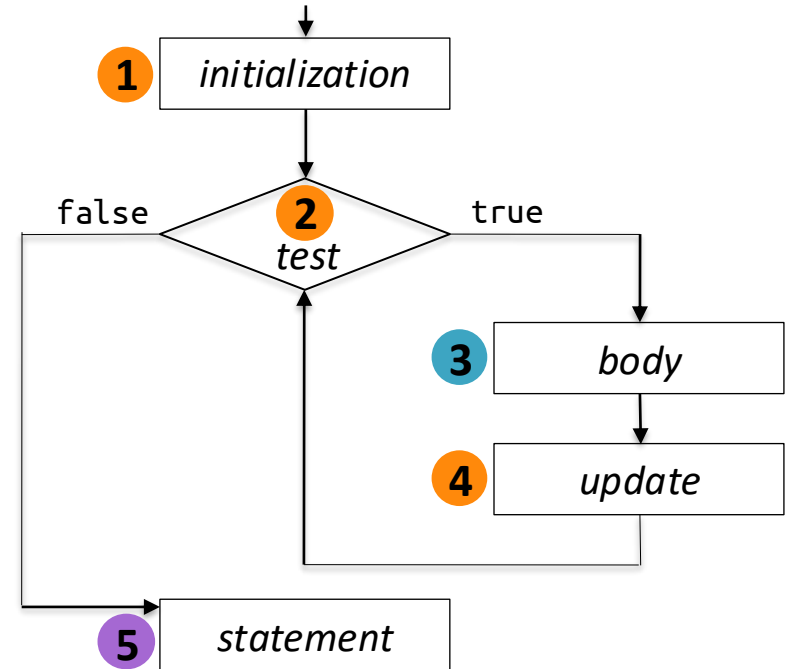
Schleifen

if / else

for-Schleife («for loop»): Kontrollfluss

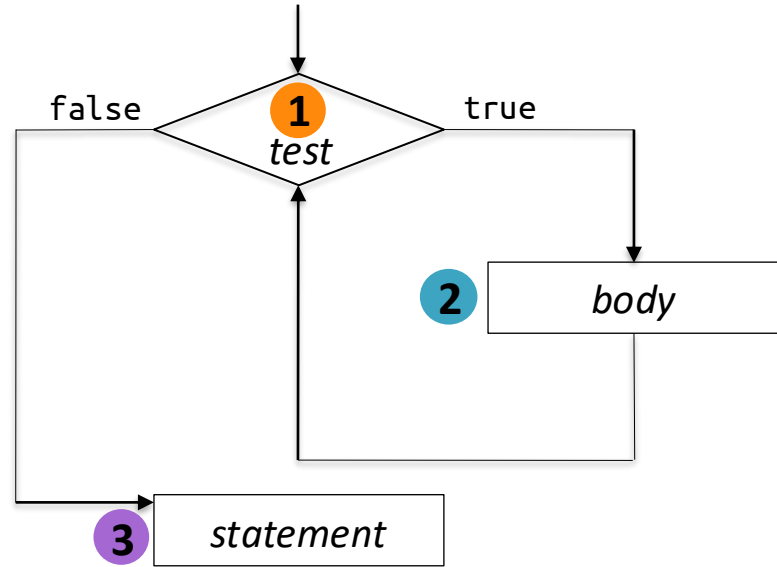
```
for (1initialization; 2test; 4update) {  
    3body  
}  
5statement;
```

```
for (int i = 1; i <= 5; i = i + 1) {  
    System.out.println(i); // 1 2 3 4 5  
}
```



while-Schleife («while loop»)

```
while (test) {  
    body  
}  
statement;
```



while-Schleife führt Rumpf so lange aus, wie *test* den Wert *true* ergibt.
In anderen Worten: die while-Schleife bricht ab, sobald *test* den Wert *false* ergibt.

Schleifen: While vs. For vs. Do-While

- **For-Loop:** Benutzen wir, wenn wir die Anzahl Iterationen bereits for der Ausführung des Loops kennen.
- **While:** Benutzen wir, wenn wir die Anzahl Iterationen nicht kennen und diese Abhängig von einer Bedingung ist.
- **Do-While:** Wird benutzt, wenn wir zuerst den Code (im Loop Body) ausführen und dann erst die Bedingung prüfen wollen.

Schleifen: For vs. While

- While und Do-While können das gleiche.
- For und While aber auch!

Schleifen: For vs. While – Beispiel 1



```
1 public class MyClass {  
2     public static void main(String[] args){  
3         for(int i = 0; i < 5; i++){  
4             System.out.println("i = " + i);  
5         }  
6     }  
7 }
```

Schleifen: For vs. While – Beispiel 1



```
1 public class MyClass {  
2     public static void main(String[] args){  
3         int i = 0;           // Initialisierung  
4         while(i < 5){        // Bedingung  
5             System.out.println("i = " + i);  
6             i++;             // Aktualisierung  
7         }  
8     }  
9 }
```

Schleifen: For vs. While – Beispiel 2



```
1 public class MyClass {
2     public static void main(String[] args){
3         int i = 1;
4         while(fancyCheck(i)){
5             int result = complexOperation(i);
6             i = i * 42 - 15;
7             System.out.println(result);
8         }
9     }
10 }
```

Schleifen: For vs. While – Beispiel 2



```
1 public class MyClass {
2     public static void main(String[] args){
3         int i = 1;
4         for(; fancyCheck(i); ){
5             int result = complexOperation(i);
6             i = i * 42 - 15;
7             System.out.println(result);
8         }
9     }
10 }
```

Schleifen: Do-While?

- Selten genutzt aber kann sehr nützlich sein!

```
1 public class MyClass {  
2     public static void main(String[] args){  
3         Scanner console = new Scanner(System.in);  
4         String pin;  
5         do {  
6             pin = console.nextLine();  
7         } while(!isPassword(pin));  
8         System.out.println("You logged in successfully!");  
9         console.close();  
10    }  
11 }
```

Inkrement und Dekrement

++i

i++

Inkrement und Dekrement

- **Präfix: ++i**

```
int a = 5;  
int b = ++a; // Zuerst wird a inkrementiert, dann wird a zugewiesen.  
  
// a = 6, b = 6
```

- **Postfix i++**

```
int a = 5;  
int b = a++; // Zuerst wird a zugewiesen, dann wird a inkrementiert.  
  
// a = 6, b = 5
```

Weitere Kurzformen

- **Prä-Inkrement** («pre-increment») und **Prä-Dekrement** («pre-decrement»):
zuerst ändern, dann verwenden

- `++variable` für `variable = variable + 1`

- `--variable` für `variable = variable - 1`

- Veränderung mit beliebigen Wert (statt nur ± 1)

- `variable += value` für `variable = variable + value;`

- `variable -= value` für `variable = variable - value;`

- `variable *= value` für `variable = variable * value;`

- `variable /= value` für `variable = variable / value;`

- `variable %= value` für `variable = variable % value;`

Achtung:

`x += 3`

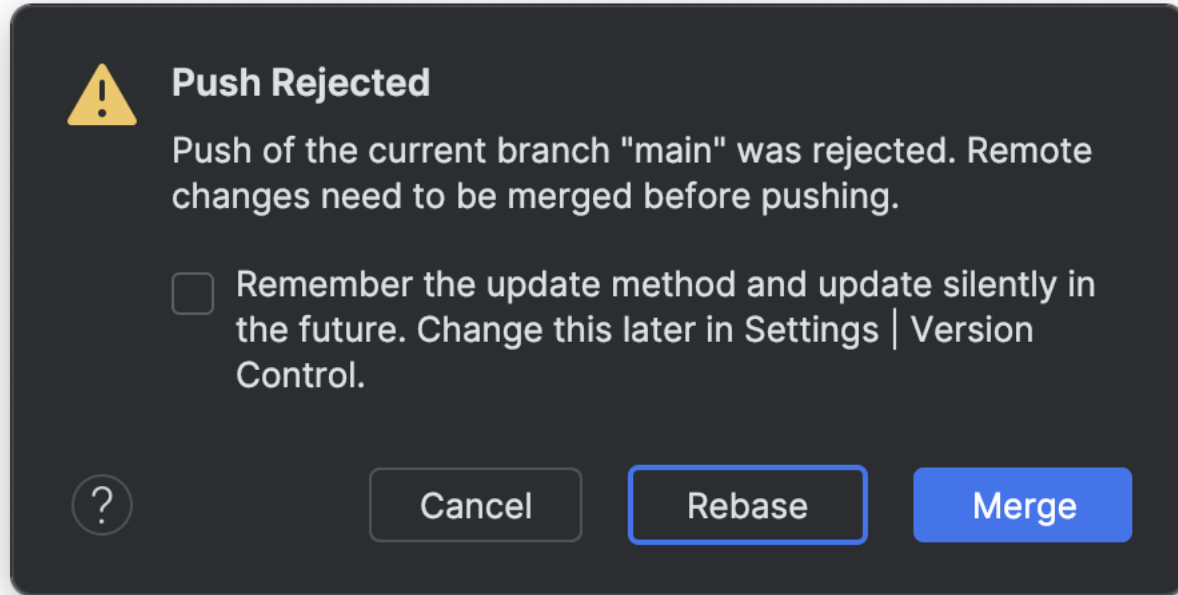
`x =+ 3`

Lesbarkeit

- **Kurzformen bedacht einsetzen**, da sie die Lesbarkeit eures Programmes erschweren
- leicht verständlicherer => schnelleres Debuggen

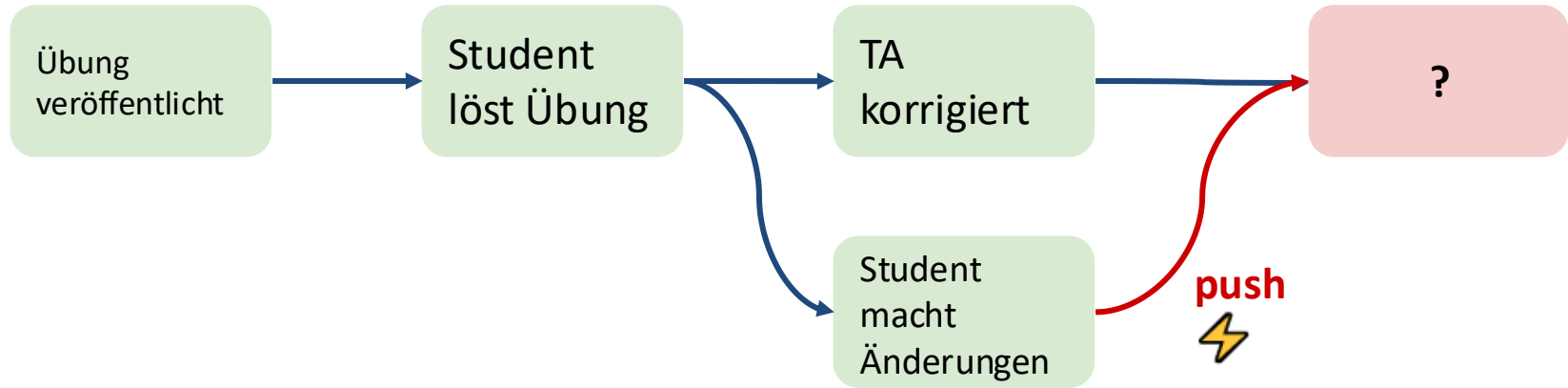
Git Merge Conflicts

Git sagt "Push Rejected"



Git - Merge Conflicts

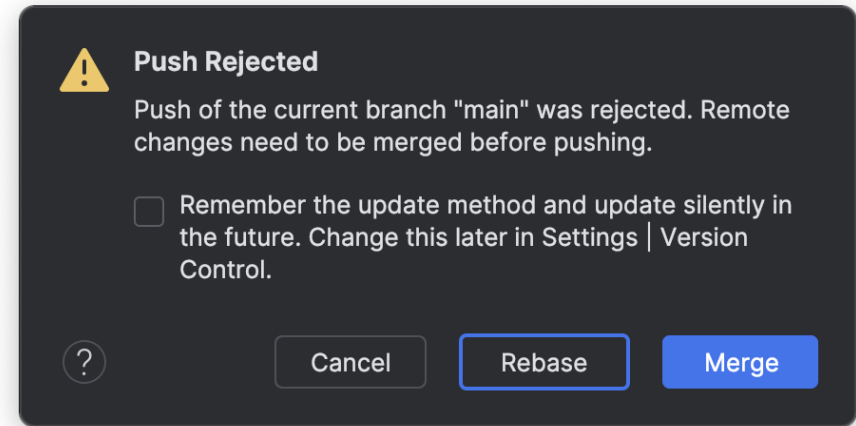
Student löst Übung gleichzeitig während TA korrigiert
→ Beim Versuch zu pushen, Konflikt!



Zwei Optionen

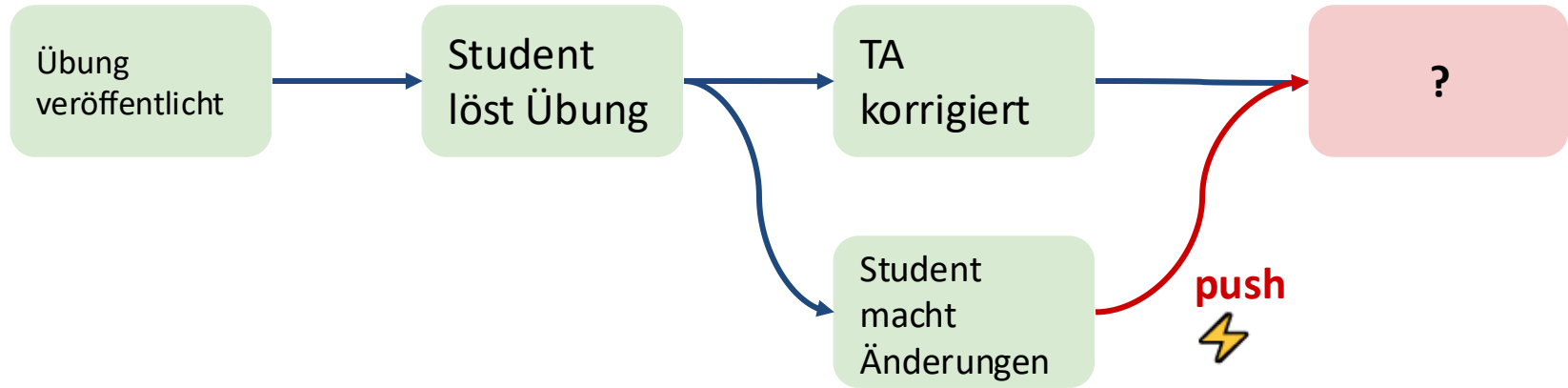
🔗 **Merge:** Kombiniert die zwei unterschiedlichen Versionen der Datei in eine kombinierte Datei.

🔗 **Rebase:** Übernimmt die Änderungen auf dem Server und versucht deine Änderungen anzuwenden.

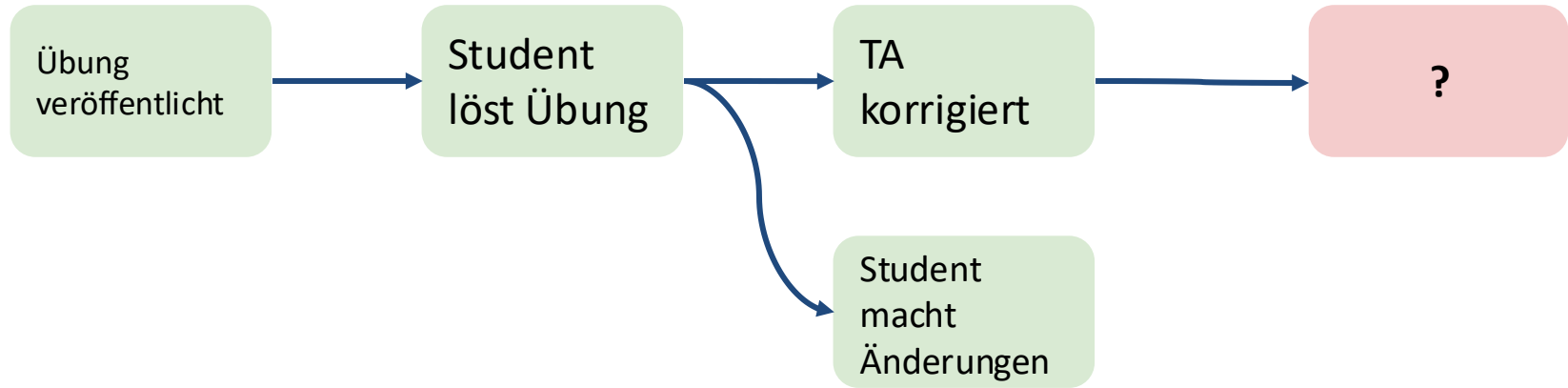


Git Merge

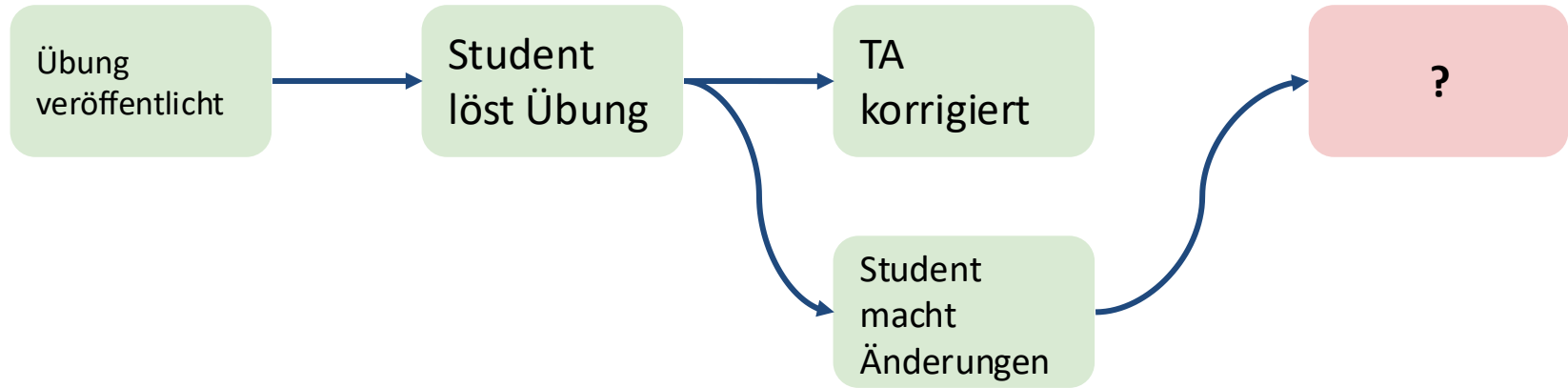
Git – Merge



Git – Merge (Accept Theirs)

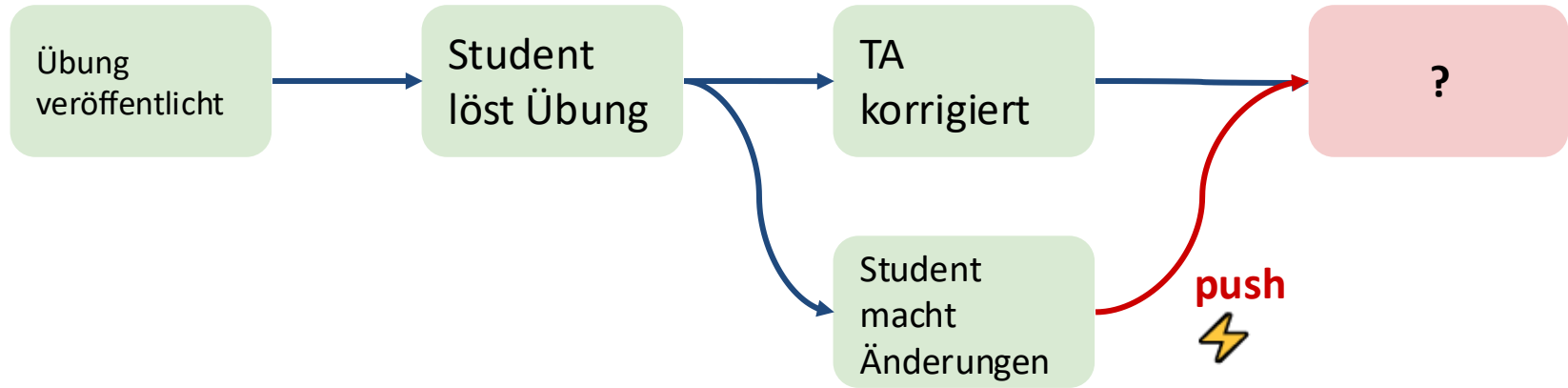


Git – Merge (Accept Yours)

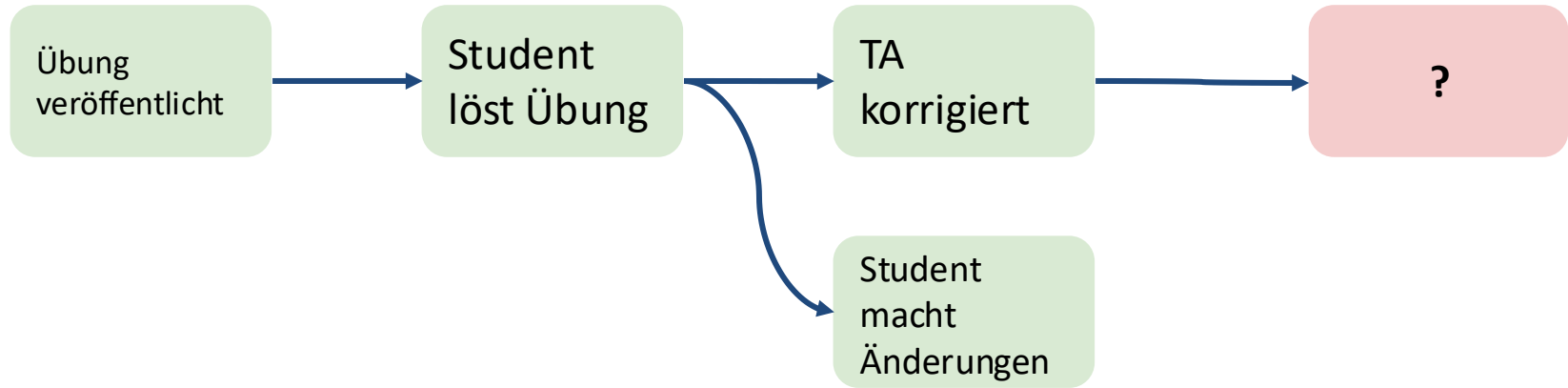


Git Rebase

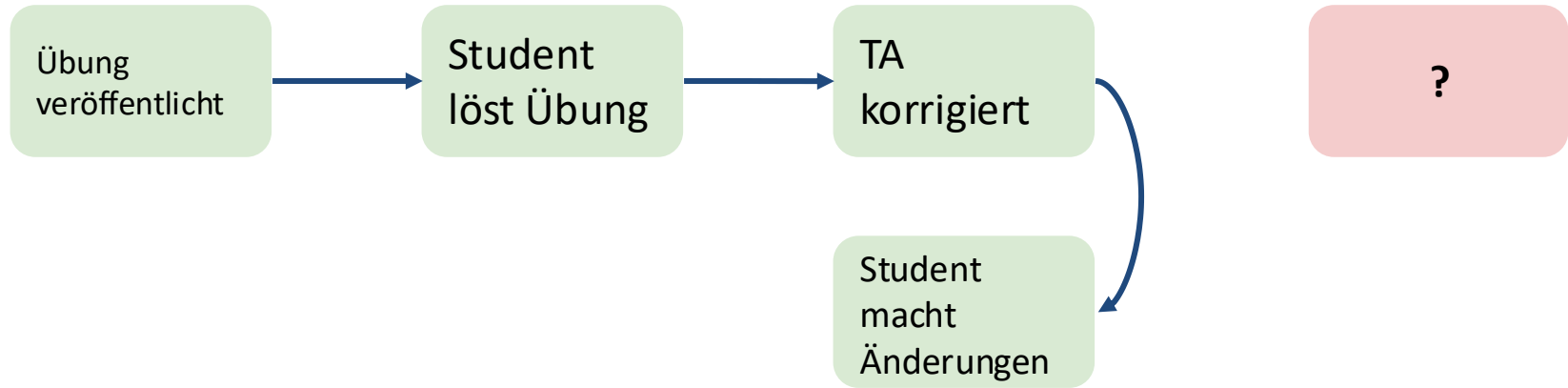
Git – Rebase



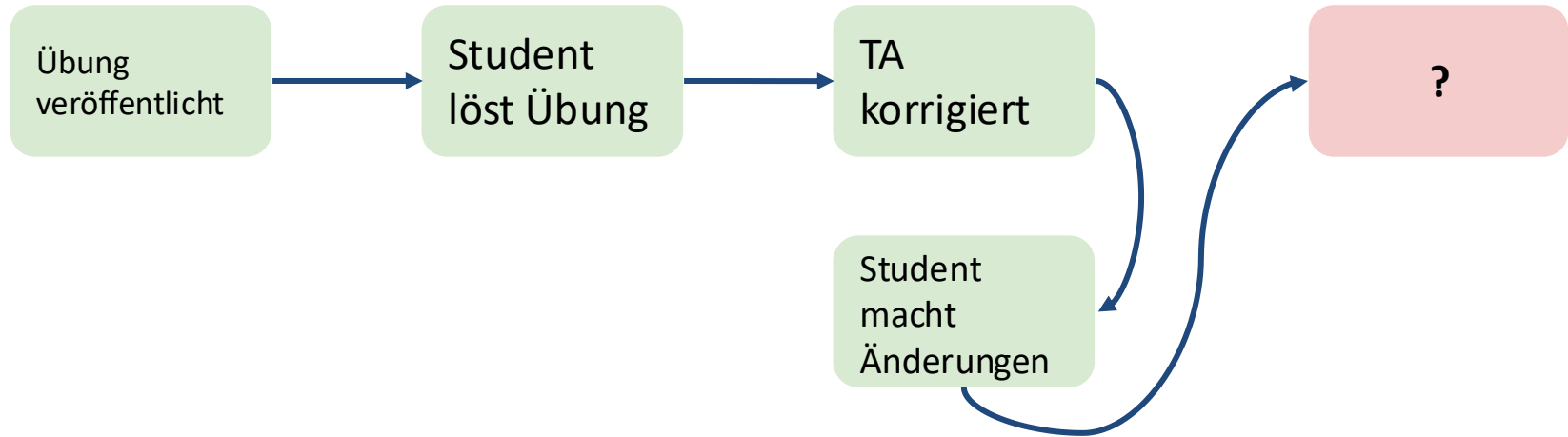
Git – Rebase



Git – Rebase



Git – Rebase



Zwei Situationen

❌ TA und Student haben nicht dieselbe(n) Datei(en) bearbeitet. 🤖

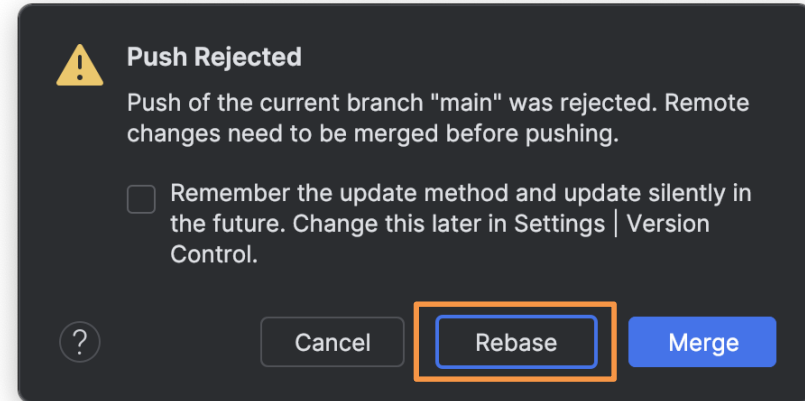
❌ TA und Student haben dieselbe(n) Datei(en) bearbeitet. 😓

Zwei Situationen

✂ TA und Student haben nicht dieselbe(n) Datei(en) bearbeitet. 🤖

✂ *Rebase* wählen.

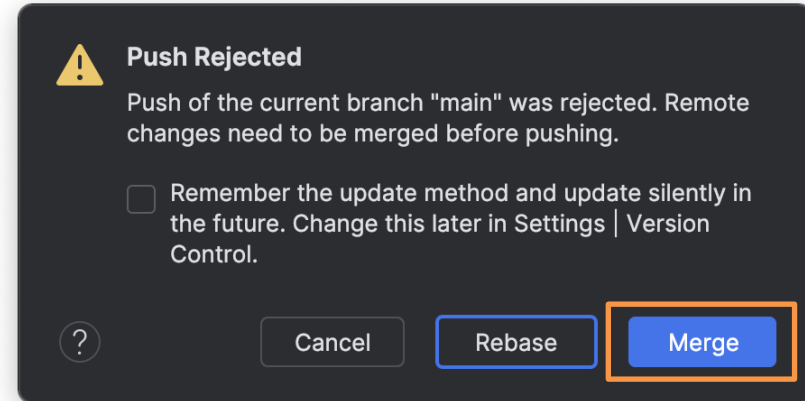
✂ Änderungen sind gepushed.



Zwei Situationen

❌ TA und Student haben dieselbe(n) Datei(en) bearbeitet. 🤔

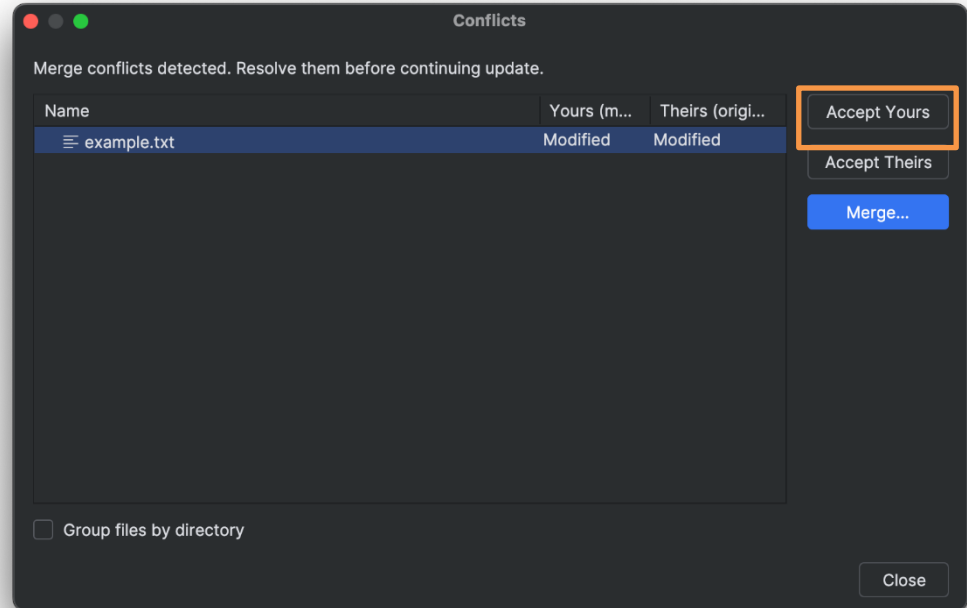
✂ *Merge* wählen.



Zwei Situationen

✂ TA und Student haben dieselbe(n) Datei(en) bearbeitet. 🤔

- ✂ *Merge* wählen.
- ✂ *Accept Yours* wählen.
- ✂ Änderungen gepushed.



Methoden

print vs return

Methoden: Folgen und Reihen

Schreiben Sie ein Programm "Reihe.java", das eine Zahl N von der Konsole einliest und dann die folgende Summe berechnet:

$$\frac{1}{1^2} + \frac{1}{2^2} + \cdots + \frac{1}{N^2}$$

Beispiel: Für $N = 4$ sollte Ihr Programm ca. 1.42 ausgeben. Wie verhält sich diese Summe für grosse N ?

cs.rohlik.net --> Eprog Übungsstunden --> Kapitel 5

JUnit Tests: Introduction

- JUnit Test erlauben es Code zu testen
- ihr habt eine Auswahl an Tests um die Funktionalität eures Programmes zu testen, es gibt weitere (hidden) tests für die Bewertung (die testen edge cases, etc.)
- Jetzt: Tests benutzen
- Später: Tests selbst schreiben

Strings

String-Methoden, die String liefern

Name der Methode	Beschreibung der Methode
substring(i1, i2) or substring(i1)	Ein neuer String: Der Substring von i1 (inklusive) bis i2 (<u>exklusive</u>); falls kein i2 übergeben wird: bis Ende des Strings
toLowerCase()	Ein neuer String mit nur Kleinbuchstaben
toUpperCase()	Ein neuer String mit nur Grossbuchstaben
stripLeading()	Ein neuer String ohne Leerzeichen am Anfang
stripTrailing()	Ein neuer String ohne Leerzeichen am Ende

Substrings

```
String name = "Bob Dylan";  
String firstName = name.substring(0, 3); // "Bob"  
String lastName = name.substring(4); // "Dylan"  
String firstCharacter = name.substring(0, 1); // "B"  
name = name.toLowerCase(); // "bob dylan"
```

Indizes 0, 1, 2

Von Index 4
bis Ende

name:	B	o	b		D	y	l	a	n
Index	0	1	2	3	4	5	6	7	8

Achtung: `name.substring(0, 1)` ist nicht das gleiche wie `name.charAt(0)`

- `name.substring(0, 1)` hat Wert "B" und ist vom Typ String
- `name.charAt(0)` hat Wert 'B' und ist vom Typ char

String-Methoden, die `int` liefern

Name der Methode	Beschreibung der Methode
<code>length()</code>	Länge des Strings (Anzahl Zeichen)
<code>indexOf(s, fromIndex)</code> <code>indexOf(c, fromIndex)</code> <code>indexOf(s)</code> <code>indexOf(c)</code>	Index wo String <code>s</code> oder Zeichen <code>c</code> zum ersten Mal nach Index <code>fromIndex</code> im String auftaucht (-1 falls nicht gefunden); falls <code>fromIndex</code> nicht übergeben wird, von Anfang des Strings

```
String s = "S. Beckett";  
int indexBeck = s.indexOf("Beck");           // 3  
int eFirstOccurrence = s.indexOf('e');        // 4  
int eSecondOccurrence = s.indexOf('e', 5);    // 7
```

String-Methoden, die boolean liefern

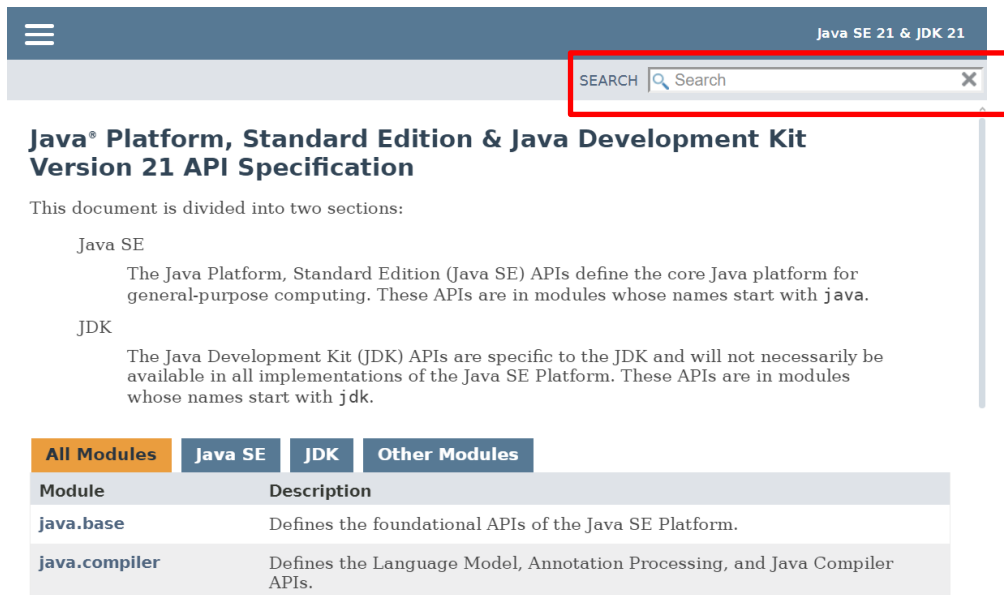
Name der Methode	Beschreibung der Methode bei Aufruf <code>s1.method(s2)</code>
<code>equals(s2)</code>	ob s1 und s2 die gleichen Buchstaben enthalten
<code>equalsIgnoreCase(s2)</code>	ob s1 und s2 die gleichen Buchstaben enthalten, ohne Berücksichtigung von Gross- und Kleinschreibung
<code>startsWith(s2)</code>	ob s1 mit den Buchstaben von s2 anfängt
<code>endsWith(s2)</code>	ob s1 mit den Buchstaben von s2 endet
<code>contains(s2)</code>	ob s1 irgendwo s2 enthält

Hinweis zu `==` und `equals`

- `==` nur für Basistypen (z.B. `int` oder `char`), nicht für Strings
- `==` für Strings wird später erklärt!

Wo findet man diese Informationen?

■ In der Java Documentation 25



Java SE 21 & JDK 21

SEARCH

Java® Platform, Standard Edition & Java Development Kit Version 21 API Specification

This document is divided into two sections:

Java SE

The Java Platform, Standard Edition (Java SE) APIs define the core Java platform for general-purpose computing. These APIs are in modules whose names start with `java`.

JDK

The Java Development Kit (JDK) APIs are specific to the JDK and will not necessarily be available in all implementations of the Java SE Platform. These APIs are in modules whose names start with `jdk`.

All Modules	Java SE	JDK	Other Modules
Module	Description		
<code>java.base</code>	Defines the foundational APIs of the Java SE Platform.		
<code>java.compiler</code>	Defines the Language Model, Annotation Processing, and Java Compiler APIs.		

-> experimentiert mit den IntelliJ Vorschlägen

String Beispiele

cs.rohlik.net --> Eprog Übungsstunden --> Kapitel 5

String Beispiele



```
1 public class MyClass{  
2     public static void main(String[] args){  
3         String str = "EPROG2024";  
4         int result = str.indexOf('2') + str.length() * 2 - 10;  
5     }  
6 }
```

Output: 13

String Beispiele



```
1 public class MyClass{
2     public static void main(String[] args){
3         String str = "Einfuehrung";
4         String result = str.substring(3, 8).toUpperCase() + str.charAt(str.length() - 1);
5     }
6 }
```

Output: FUEHRg

String Beispiele



```
1 public class MyClass{
2     public static void main(String[] args){
3         String str = "Programmierung";
4         String result = str.replace('r', '*').substring(5, 12) + str.charAt(0);
5     }
6 }
```

Output: ammie*uP

String Beispiele



```
1 public class MyClass{
2     public static void main(String[] args){
3         String str1 = "Java21";
4         String str2 = "EPROG2024";
5         String result = str1.substring(0, 2) + str2.substring(str2.length() - 4);
6     }
7 }
```

Output: Ja2024

String Beispiele



```
1 public class MyClass{
2     public static void main(String[] args){
3         String str = "Substring";
4         int result = str.indexOf('S') + str.lastIndexOf('g') - str.length();
5     }
6 }
```

Output: -1

String Beispiele



```
1 public class MyClass{
2     public static void main(String[] args){
3         String str = "Einfuehrung";
4         String result = str.substring(2, 6).replace('f', 'X').concat(str.substring(6, 9));
5     }
6 }
```

Output: nXuehru

String Beispiele

```
1 public class MyClass{  
2     public static void main(String[] args){  
3         String str = "Programmierung";  
4         String result = str.charAt(0) + str.substring(2, 5).toLowerCase() + str.charAt(str.length() - 1);  
5     }  
6 }
```

Output: Pogrg

Vorbesprechung

Aufgabe 2: Grösster gemeinsamer Teiler



Aufgabe 2: Grösster gemeinsamer Teiler

Schreiben Sie ein Programm “GGT.java”, das den grössten gemeinsamen Teiler (ggT) zweier positiver ganzer Zahlen mithilfe des Euklidischen Algorithmus berechnet. Hierbei handelt es sich um eine iterative Berechnung, die auf folgender Beobachtung basiert:

1. Wenn x grösser oder gleich y ist und sich x durch y teilen lässt, dann ist der ggT von x und y gleich y ;
2. andernfalls (wenn x kleiner als y ist oder sich x nicht durch y teilen lässt) speichern Sie zuerst den Rest $r = x \% y$. Setzen Sie anschliessend x auf den bisherigen Wert von y und y auf r und wiederholen Sie den Algorithmus.

Aufgabe 1: Binärdarstellung

Schreiben Sie ein Programm “Binaer.java”, das eine positive Zahl Z einliest und dann die Binärdarstellung druckt. (Hinweis: Finden Sie zuerst die grösste Zahl k , so dass die Zweierpotenz $K = 2^k$ kleiner als Z ist.)

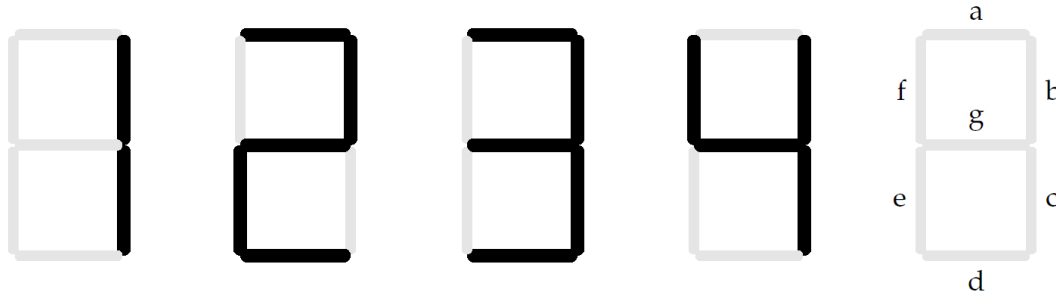
Beispiel: Für $Z = 14$ sollte Ihr Programm 1110 ausgeben.

Aufgabe 3: Zahlenerkennung

Für diese Aufgabe verwenden wir einen String um die erleuchtenden Segmente einer [Sieben-segmentanzeige](#) zu kodieren. Die Segmente sind, wie im Bild gezeigt, von a bis g nummeriert. Die Kodierung einer möglichen Anzeige ist ein String, in welchem der Buchstabe 'x' genau dann vorkommt, wenn das 'x'te Segment der Anzeige erleuchtet ist. Zum Beispiel wird die Zahl 2 kodiert durch 'abged'. Zur Einfachheit darf angenommen werden, dass kein Buchstabe mehr als einmal in der Kodierung vorkommt und dass nur die Zahlen 0 bis 9 kodiert werden.

Schreiben Sie ein Programm "Zahlen.java", das einen String, der eine Anzeige kodiert, einliest und die kodierte Zahl als Integer ausgibt. Überlegen Sie wie viele IF Blöcke benötigt werden um jede Zahl zu erkennen.

Tip: Sie können `str.contains("a")` verwenden, um zu überprüfen, ob ein String `str` den Buchstaben 'a' enthält.



Aufgabe 4: Berechnungen



2. Implementieren sie die Methode `Calculations.magic7(int a, int b)`. Die Methode gibt einen Boolean zurück. Die Methode soll `true` zurückgeben, wenn einer der Parameter 7 ist oder wenn die Summe oder Differenz der Parameter 7 ist. Ansonsten soll die Methode `false` zurückgeben.

Beispiele

- `magic7(2,5)` gibt `true` zurück.
- `magic7(7,9)` gibt `true` zurück.
- `magic7(5,6)` gibt `false` zurück.

Hinweis: Mit der Funktion `Math.abs(num)` können Sie den absoluten Wert einer Zahl `num` erhalten.

Aufgabe 4: Berechnungen



3. Implementieren Sie die Methode `Calculations.fast12(int z)`. Das Argument `z` ist nicht negativ. Die Methode gibt einen Boolean zurück. Die Methode soll `true` zurückgeben, wenn `z` nahe an einem Vielfachen von 12 ist. Eine Zahl `x` ist nahe an einer Zahl `y`, wenn eine der Zahlen um maximal 2 grösser oder kleiner ist als die andere Zahl. Ansonsten soll die Methode `false` zurückgeben.
- `fast12(12)` gibt `true` zurück.
 - `fast12(14)` gibt `true` zurück.
 - `fast12(10)` gibt `true` zurück.
 - `fast12(15)` gibt `false` zurück.

Aufgabe 5: Scrabble



In dieser Aufgabe sollen Sie Scrabble-Steine legen, mittels ASCII-Art auf der Konsole. Vervollständigen Sie die Methode `drawNameSquare` in der Klasse `Scrabble`. Diese Methode nimmt einen Namen als `String`-Parameter und soll den Namen als in einem Quadrat angeordnete Scrabble-Steine auf der Konsole (`System.out`) ausgeben. Wenn z.B. der `String` `Alfred` übergeben wird, sollte folgendes Bild ausgegeben werden:

```
+---+---+---+---+---+---+
| A | L | F | R | E | D |
+---+---+---+---+---+---+
| L |                               | E |
+---+                               +---+
| F |                               | R |
+---+                               +---+
| R |                               | F |
+---+                               +---+
| E |                               | L |
+---+---+---+---+---+---+
| D | E | R | F | L | A |
+---+---+---+---+---+---+
```

Aufgabe 6 und 7: Operatoren



Aufgabe 6: Operatoren: Grundlagen

In dieser Aufgabe untersuchen Sie, in welcher Reihenfolge Java die Bestandteile eines Ausdrucks auswertet. Beachten Sie dabei insbesondere die Priorität und Assoziativität der Operatoren, Ganzzahldivision, String-Verkettung, Typumwandlungen und die Kurzschlussauswertung boolescher Ausdrücke. Die folgenden Codefragmente sind voneinander unabhängig und werden jeweils innerhalb einer `main`-Methode ausgeführt.

Bestimmen Sie für jede Teilaufgabe die exakte Konsolenausgabe, ohne den Code auszuführen. Berücksichtigen Sie dabei auch Leerzeichen, Zeilenumbrüche und den Datentyp des Ergebnisses. Kennzeichnen Sie jedes Leerzeichen in Ihrer Antwort explizit mit `_`. Beginnt die Konsolenausgabe eine neue Zeile, beginnen Sie auch in Ihrer Antwort eine neue Zeile. Sie dürfen Ihre Antworten anschliessend durch Ausführen des Codes kontrollieren.

a) `System.out.println(18 - 3 * 4 + 17 % 5);`

Ausgabe: _____

Aufgabe 7: Operatoren: Vertiefung

In dieser Aufgabe kombinieren Sie mehrere Regeln zur Auswertung von Java-Ausdrücken. Achten Sie zusätzlich auf Klammerung, gemischte Ganzzahl- und Gleitkommaarithmetik, Präfix- und Postfix-Inkremente, den bedingten Operator, Zuweisungen, Casts sowie die Darstellung von `double`-Werten. Wie in Aufgabe 6 sind alle Codefragmente voneinander unabhängig und werden jeweils innerhalb einer `main`-Methode ausgeführt.

Bestimmen Sie wiederum die exakte Konsolenausgabe, ohne den Code auszuführen. Kennzeichnen Sie jedes Leerzeichen mit `_` und übernehmen Sie jeden Zeilenumbruch der Konsolenausgabe in Ihre Antwort.

a) `System.out.println(48 / (4 / 2) + 7 % 4);`

Ausgabe: _____

Nach- besprechung

Aufgabe 4: Berechnungen

1. Vervollständigen Sie "SharedDigit.java". In der main-Methode sind zwei int-Variablen a und b deklariert und mit einem Wert zwischen 10 und 99 (einschliesslich) initialisiert. Das Programm soll einer int-Variablen r einen bestimmten Wert zuweisen. Wenn a und b eine Ziffer gemeinsam haben, dann wird r die gemeinsame Ziffer zugewiesen (wenn a und b beide Ziffern gemeinsam haben, dann kann eine beliebige Ziffer zugewiesen werden). Wenn es keine gemeinsame Ziffer gibt, dann soll -1 zugewiesen werden. Sie brauchen für dieses Programm keine Schleife.

Beispiele:

- Wenn a: 34 und b: 53, dann ist r: 3
- Wenn a: 10 und b: 22, dann ist r: -1
- Wenn a: 66 und b: 66, dann ist r: 6
- Wenn a: 34 und b: 34, dann ist r: 3 oder 4

Testen Sie Ihre Lösung mit a gleich 34 und b gleich 43. Was liefert Ihr Programm?

2. Vervollständigen Sie "SumPattern.java". In der main-Methode sind drei int-Variablen a, b und c deklariert und mit irgendwelchen Werten initialisiert. Wenn die Summe von zwei der Variablen die dritte ergibt, nehmen wir an, dass $a + c == b$, so soll die Methode "Moeglich. $a + c == b$ " ausgegeben (wobei die Werte für a, b und c einzusetzen sind). Wenn das nicht der Fall ist, dann soll die Methode "Unmoeglich." ausgegeben.

Beispiele:

- Wenn a: 4, b: 10, c: 6, dann wird "Moeglich. $4 + 6 == 10$ " oder "Moeglich. $6 + 4 == 10$ " ausgegeben.
- Wenn a: 2, b: 12, c: 0, dann wird "Unmoeglich." ausgegeben.

3. Vervollständigen Sie "AbsoluteMax.java". In der main-Methode sind drei int-Variablen a, b und c deklariert und mit irgendwelchen Werten initialisiert. Das Programm soll einer int-Variable r den grössten absoluten Wert von a, b und c zuweisen.

Danke!